



A
Project Report
on
Unified Code — Real-Time Collaborative Code Editor
and Whiteboard

submitted as partial fulfilment for the award of

BACHELOR OF TECHNOLOGY
DEGREE

SESSION 2025-26

in

Computer Science And Engineering

by

Yash Kumar Aggarwal (2200290100195)

Dishant Rajput (2200290100063)

Vijay Kumar Chaurasiya (2200290100191)

Vishnu Kumar Verma (2200290100193)

Under the supervision of

Dr. Preeti Garg

KIET Group of Institutions, Ghaziabad

Affiliated to

Dr. A.P.J. Abdul Kalam Technical University, Lucknow
(Formerly UPTU)

May, 2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text..

Roll No.	Name	Signature	Date
2200290100195	Yash Kumar Aggarwal		23-04-26
2200290100063	Dishant Rajput		23-04-26
2200290100191	Vijay Kumar Chaurasiya		23-04-26
2200290100193	Vishnu Kumar Verma		23-04-26

CERTIFICATE

This is to certify that the project report entitled “**Unified Code — Real-Time Collaborative Code Editor and Whiteboard**” has been successfully completed and submitted by **Yash Kumar Aggarwal, Dishant Rajput, Vijay Kumar Chaurasiya, and Vishnu Kumar Verma** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Department of Computer Science and Engineering**.

The project work has been carried out by the students under my supervision during the academic session **2025–26** at the **Department of Computer Science and Engineering, KIET Group of Institutions, Ghaziabad**, affiliated to **Dr. A.P.J. Abdul Kalam Technical University, Lucknow**.

To the best of my knowledge, the work presented in this report is original and has not been submitted elsewhere for the award of any other degree or diploma. The students have demonstrated sincere effort, technical understanding, and dedication in the successful completion of this project. Their work reflects a good understanding of modern software development practices, particularly in the domain of real-time collaborative systems, web technologies, and distributed communication architectures.

Dr. Preeti Garg

**Associate Professor
Dean(CSE)**

Dr. Vineet Sharma

Senior Professor &

Date: 23-04-26

ACKNOWLEDGEMENT

It gives us great sense of pleasure to present this project report on “**Unified Code — Real-Time Collaborative Code Editor and Whiteboard**”, undertaken as part of our final year Bachelor of Technology in Computer Science and Engineering.

We would like to express our sincere gratitude to our project guide, **Dr. Preeti Garg**, for her continuous support, valuable guidance, and encouragement throughout the development of this project. Her insightful suggestions and constructive feedback played a crucial role in shaping our understanding of real-time systems and modern web technologies. Her motivation helped us overcome challenges and complete this work successfully.

We also take the opportunity to acknowledge the contribution of Dr. Vineet Sharma, Dean of the Department of Computer Science & Engineering, KIET, Ghaziabad, for his full support and assistance during the development of the project. We also do not like to miss the opportunity to acknowledge the contribution of all the faculty members of the department for their kind assistance and cooperation during the development of our project.

We also do not like to miss the opportunity to acknowledge the contribution of all faculty members, especially faculty/industry person/any person, of the department for their kind assistance and cooperation during the development of our project. Last but not the least, we acknowledge our friends for their contribution in the completion of the project.

Roll No.	Name	Signature	Date
2200290100195	Yash Kumar Aggarwal		23-04-26
2200290100063	Dishant Rajput		23-04-26
2200290100191	Vijay Kumar Chaurasiya		23-04-26
2200290100193	Vishnu Kumar Verma		23-04-26

ABSTRACT

In the era of distributed software development and remote work, the demand for real-time collaborative platforms for developers is on the rise. Current development practices involve the use of various tools for code editing, communication and visualisation, resulting in productivity loss from switching between tools. This project aims to tackle these problems with "Unified Code: Real-Time Collaborative Code Editor and Whiteboard", a web application that combines various collaborative tools into a single platform.

The system design offers real-time collaborative code editing, file synchronization, integrated chat feature to communicate text-based ideas, and a collaborative whiteboard to visually represent ideas. The system is built with modern web framework stack, React.js for frontend, CodeMirror 6 for code editing and tldraw for whiteboard. IO is used for the backend, which is developed with Node.js and Express.js.

The system supports collaboration through rooms, enabling users to join a room and work together in real-time. Every user interaction, including code editing, file management, chat messaging and drawing, is transparently coordinated via server events. Furthermore, the system provides multi-language code execution ability by integrating with external services, enabling secure code compilation and execution without installation.

Experimental results show the system has low latency and good synchronization between users and is applicable in other scenarios, such as pair programming, technical interviews, and distance learning. The system's design focuses on simplicity, modularity, and efficiency for easy integration and scalability.

Overall, Unified Code gives a collective approach of collaborative programming improving developer productivity.

Keywords:

Real-Time Collaborations, Code Editors, Socket.IO, WebSockets, Collaborative Systems, React.js, CodeMirror, Whiteboard, Distributed Systems.

TABLE OF CONTENTS

DECLARATION	ii
CERTIFICATE	III
ACKNOWLEDGEMENT	IV
ABSTRACT	V
TABLE OF CONTENTS	VI
LIST OF FIGURES	IX
LIST OF TABLES	X
LIST OF ABBREVIATIONS	XI
CHAPTER 1 INTRODUCTION	1
1.1 Introduction	1
1.2 Background And Context	1
1.3 Problem Statement	2
1.4 Objectives Of The Project	4
1.4.1 Development Of A Real-Time Collaborative Code Editor	4
1.4.2 Integration Of Real-Time Communication System	4
1.4.3 Implementation Of A Collaborative Whiteboard	4
1.4.4 Design Of A File Management System	4
1.4.5. Integration Of Multi-Language Code Execution	5
1.4.6 Implementation Of Real-Time Synchronization Mechanism	5
1.4.7 Ensuring System Performance And Usability	5
1.5 Scope Of The Project	5
1.5.1 Functional Scope	5
1.5.2 Technical Scope	5
1.5.3 Application Scope	6
1.5.4 Limitations	6
1.6 Significance Of The Work	7

CHAPTER 2 LITERATURE REVIEW	8
2.1 Introduction To The Literature	8
2.2 Existing Collaborative Systems	9
2.2.1 Cloud-Based Integrated Development Environments (Ides)	10
2.2.2 Real-Time Code Collaboration Tools	10
2.2.3 Document-Based Collaboration Systems	10
2.2.4 Communication And Whiteboard Tools	10
2.2.5. Limitations Of Existing Systems	11
2.3 Real-Time Synchronization Techniques	11
2.4 Communication Technologies (Web Sockets)	13
2.4.1. 1. Inefficiency Of Traditional Methods	13
2.4.2 Websocket Protocol	13
2.4.3. Socket.Io Framework	13
2.4.4. Role In Real-Time Collaboration	14
2.4.5. Challenges	14
CHAPTER 3 SYSTEM DESIGN & METHODOLOGY	15
3.1 SYSTEM OVERVIEW	15
3.1.1 Client Side	15
3.1.2 Servier Side	15
2.1.3. External Services	15
3.1.4. Communication Model	16
3.1.5. Key Features of System	16
3.1.6. Design Priniciples	16
3.2 ARCHITECTURE DESIGN	17
3.3 CLIENT LAYER ARCHITECTURE.	18
3.3.1 Component Architecture	18
3.1.2 Code Mirror 6 Integration	18

3.4 SERVER LAYER ARCHITECTURE	20
3.5 REAL-TIME COLLABORATION MODEL	22
3.5 COLLABORATIVE CODE EDITING IMPLEMENTATION	24
3.6 FILE MANAGEMENT SYSTEM	26
3.7 MULTI-LANGUAGE CODE EXECUTION	26
3.8 INTEGRATED TEXT CHAT	27
3.9 INTEGRATED TEXT CHAT	27
 CHAPTER 4 IMPLEMENTATION & RESULT	 28
4.1 INTRODUCTION	28
4.2 REAL-TIME SYNCHRONIZATION LATENCY	28
4.2 CONSISTENCY VERIFICATION	29
4.3 RESOURCE UTILIZATION	29
4.4 FEATURE COMPARISON	30
 CHAPTER 5 CONCLUSION & SCOPE	 31
5.1 CONCLUSION	31
5.2 FUTURE SCOPE	32
5.2.1 SCALABILITY ENHANCEMENTS	32
5.2.2 STORAGE AND VERSION CONTROL	32
5.2.3 ADVANCED COLLABORATION	32
5.2.4 IMPROVISATION IN CODE EDITOR	33
5.2.5 SECURITY IMPROVISATIONS	33
5.2.5 EXTERNAL TOOLS	33
 REFERENCES	 34

LIST OF FIGURES

Figure No.	Description	Page No.
3.1	Overall System Architecture of Unified Code	16
3.2	Layered Architecture of Unified Code System	20
3.3	Real-Time Collaboration Event Flow	24

LIST OF TABLES

Table No.	Description	Page No.
1	Latency Comparison Table	28
2	Consistency Measuring Table	29
3	Client Side Resource Utilisation Table	29
4	Server Side Resource Utilisation Table	29
5	Feature Comparison Table	

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
AST	Abstract Syntax Tree
CRDT	Conflict-Free Replicated Data Type
CSS	Cascading Style Sheets
DOM	Document Object Model
HTTP	Hyper Text Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JS	JavaScript
LSP	Language Server Protocol
Node.js	JavaScript Runtime Environment
OT	Operational Transformation
REST	Representational State Transfer
UI	User Interface
UX	User Experience
CDN	Content Delivery Network
CPU	Central Processing Unit
RAM	Random Access Memory

CHAPTER 1

INTRODUCTION

1.1 Introduction

Modern software development increasingly demands real-time collaboration, yet developers rely on different tools, separate applications for code editing, messaging, video calls, and diagramming etc.. This creates overhead through constant context switching, particularly during pair programming, technical interviews, and remote teamwork. Existing solutions include cloud IDEs that require heavy installations with limited communication features with browser platforms that treat collaboration as secondary. Meanwhile, collaborative editors like Google Docs prove that seamless multi-user editing is possible but it does not address the needs of software development. UnifiedCode has four critical capabilities into a single browser-based workspace. It has real-time multi-user code editing with syntax highlighting and cursor awareness across multiple files, integrated text chat for seamless communication within the coding context, collaborative whiteboard for visualizing algorithms and system architectures. It also has multi-language code execution that supports 40+ languages through external compiler APIs. The project uses React.js for the frontend, CodeMirror 6 for editing, tldraw for vector drawing, and Node.js with Socket.IO for real-time communication.

1.2 Background and Context

The evolution of software development methodologies has led to a shift in how developers work together to develop software. Over the past few years, there has been a shift from the traditional, co-located software development process to the remote and distributed process. This is due to the growth of global software development teams, distributed workplaces and the demand for real-time collaboration in development processes. This has led to an increased use of various digital tools for developers, including code editors, communication tools, and

whiteboard tools.

But there are inefficiencies in the coding process when using multiple tools for code editing, communication and visualization. This can lead to having to switch between code editors, instant messaging clients and whiteboards, increasing context switching, and thus reducing productivity. This is particularly evident in collaborative activities such as pair programming, whiteboarding, technical interviewing, collaborative debugging and online teaching and learning activities.

Existing methods address some of these problems. Online Integrated Development Environments (IDEs) provide online collaborative coding, but can be hard to set up and lack communication support.

Web platforms are readily accessible, but may not support all real-time collaboration features. Although document collaboration systems show successful real-time editing features, they are not optimised to work with source code editing, featuring syntax highlighting, structured text formatting and multiple file editing.

To address these constraints, a solution where all the tools are integrated into a single platform is needed. The idea of a real-time collaborative workspace integrates code editing, communication and visualisation in a single system, minimising the need for several applications. These platforms use advanced web technologies, such as WebSockets and event-based systems, to inform users of each other's actions in the system.

The Unified Code platform is created to address these issues. It offers a web-based interface for real-time collaborative code editing, chat, file system and whiteboard. Its integration of these tools seeks to increase productivity, communication and efficiency in the development process in contemporary software engineering practices.

1.3 Problem Statement

With more and more development happening remotely in the software industry, developers are turning to numerous tools to help them perform tasks such as coding, communication, collaboration and visualisation. While these tools all work well, the fact they are all separate introduces several issues.

A notable problem is multitasking, in which developers are often required to switch among various tools, such as code editors, messaging tools and whiteboard tools.

A critical issue is the lack of real-time synchronization between tools. Although some tools support collaborative editing, they don't support communication or visualisation. For instance, conventional collaborative editors support mainly text synchronization but not other features such as structured code editing, multiple file management, or whiteboard sharing. The consequence of this is that users need to interact with multiple systems.

Furthermore, current solutions pose difficulties in installation and deployment. Cloudbased solutions may require setup, configuration or dependency installation, which are not ideal for ad hoc collaboration. Alternatively, minimalistic browserbased tools might lack some of the features needed for effective software development.

On a technical level, maintaining consistent state across users also poses a problem.

Simultaneous editing and manipulation of code or other resources by multiple users can lead to conflicts, inconsistency and information loss.

While sophisticated mechanisms such as Operational Transformation (OT) and Conflict-Free Replicated Data Types (CRDTs) can be used, they add further complexity to the system's design and maintenance.

Also, current systems may not offer an integrated platform supporting code execution, real-time communication, and visual collaboration in a single platform. This can be problematic in applications like collaborative programming interviews, pair programming and online education.

Thus, the main challenge we seek to address in this project is the lack of an integrated, efficient and real-time collaborative platform that supports code editing, communication, file management and visualization in one platform. The objective is to create a system that reduces context switching, enables real-time collaboration and provides a user-friendly experience for parallel software development.

1.4 Objectives of the Project

The key problem addressed by this project is the lack of an integrated, efficient, real-time collaborative system that provides code editing, communication, file management and visualization. We want to create a system that reduces context switching, enables real-time collaboration and provides an intuitive development experience in a concurrent environment.

To achieve this goal, the following specific objectives have been identified:

1.4.1 Development of a Real-Time Collaborative Code Editor

- i. To create a collaborative code editor to enable multiple users to edit code in real time.
- ii. To provide syntax highlighting, cursor movements and live updates to other users.
- iii. To achieve minimal lag and accurate synchronisation in the collaborative process.

1.4.2 Integration of Real-Time Communication System

- i. To create an in-built chat system that allows users to chat while they code.
- ii. To facilitate real-time messaging, typing notifications, and presence notifications.
- iii. To facilitate collaboration through communication while coding.

1.4.3 Implementation of a Collaborative Whiteboard

- i. To offer a collaborative whiteboard for visualizing concepts, algorithms and system designs.
- ii. To allow concurrent drawing sharing.
- iii. To offer a range of drawing tools and shapes for drawing.

1.4.4 Design of a File Management System

- i. To implement a file system which enables users to work with multiple files and directories.
- ii. To facilitate file creation, deletion, renaming and switching between files.
- iii. To maintain real-time consistency of the file system across the users.

1.4.5. Integration of Multi-Language Code Execution

- i. To allow users to compile and run code in the tool.
- ii. To allow users to run code in different languages via secure APIs.
- iii. To provide real-time output, error and performance information.

1.4.6 Implementation of Real-Time Synchronization Mechanism

- i. To implement an event-driven system, using technologies such as WebSockets and Socket.IO.
- ii. To provide a single state for all users by handling concurrent transactions.
- iii. To keep the system responsive and stable in the face of many users.

1.4.7 Ensuring System Performance and Usability

- i. To enhance the responsiveness and reduce the latency.
- ii. To create a user-friendly interface.
- iii. To make it cross-platform compatible.

1.5 Scope of the Project

The goal of the Unified Code - Real-Time Collaborative Code Editor and Whiteboard project is to create a web solution that allows users to communicate in a single integrated platform. The system is aimed at fulfilling the key requirements of collaborative software engineering by providing coding, communication and visualization..

1.5.1 Functional Scope

Our project is a coding platform that combines several features to enable collaboration and productivity, offers a collaborative code editor that supports multiple collaborators, syntax highlighting, multiple cursors and real-time changes for collaboration. It has a shared whiteboard where users can draw diagrams, illustrate algorithms and ideas.

It also involves a real-time chat system that allows users within the same workspace to communicate in real-time without relying on third-party chat apps. A file management system is also implemented to facilitate joint file/folder creation, editing, deletion and organisation. For enhanced functionality, the system also comprises a code execution component that supports the execution of code in various languages via external APIs, with the output and errors displayed in the system.

1.5.2. Technical Scope

The project is technically realised with the following technologies:

Frontend includes React.js using component-based architecture to create a user interface, CodeMirror 6 for code editing and tldraw for whiteboard, Node.js and Express.js for server-side functionalities and for Real-Time Communication, Socket.IO for event-based, two-way communication among users.

Our system is built on a client-server architecture with event-based real-time synchronization for effective communication and state synchronization.

1.5.3. Application Scope

The platform is designed to support various real-world use cases; Collaborative software development among distributed teams, Pair programming sessions, Technical Interviews and coding assessments, Online teaching and learning environments

1.5.4. Limitations

Although the system offers basic collaboration capabilities, the following are beyond the system's capabilities:

- i. Support for data persistence over time and version control
- ii. Access control (AC) and authentication
- iii. Large-scale testing with high network traffic
- iv. Connecting to external tools such as Git or CI/CD

1.6 Significance of the Work

The creation of Unified Code - a Real-Time Collaborative Code Editor and Whiteboard - is very important in today's software development landscape, with increasing amounts of remote working and distributed development teams. In today's world, developers require the ability to work together in real time, and this project provides that by integrating several important functionalities.

This project is of the great benefit for developers to be more productive as it eliminates the need of multiple tools. Instead of moving from one tool to another (to code, communicate, and share ideas), they can do this in one system. This helps developers to be more productive, efficient and effective.

This is a highly collaborative system, multiple users can collaborate. Users can interactively share the process of coding, debugging and problem solving. This is a great system for pair programming, remote software development and team work. It is also synchronised in real-time to keep everyone in the loop and support collaboration and coordination.

Unified Code also makes an excellent tool for education. It provides a collaborative learning environment for students and teachers to code together, chat, and demonstrate ideas on the whiteboard. This greatly facilitates its use in coding classes, webinars and tutoring sessions, by offering a more interactive and practical learning environment.

Lastly, this project is also a good platform for enhancement. Further enhancements to the system, such as version control, session management and advanced conflict resolution algorithms such as Operational Transformation (OT) or Conflict-free Replicated Data Types (CRDT) can be added in subsequent stages. In conclusion, the Unified Code project is a step towards efficient, accessible and modern collaborative IDEs..

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction to the Literature

The development of collaborative software engineering tools is the culmination of decades of research into distributed collaboration systems and computer communication. Understanding this historical evolution is essential for the design of UnifiedCode, as it also reveals existing gaps that drive ongoing innovation. The theoretical foundations of real-time collaborative editing date back to the late 1980s, when researchers investigated methods for multiple users to simultaneously modify shared documents without conflicts. Sun et al. formalized transformation functions and established conditions to ensure convergence and consistency.

Operational Transformation (OT) became the basis for commercial collaborative editors like Google Docs, though its complexity has posed challenges for developers over the years . In the 2000s, an alternative approach called Conflict-free Replicated Data Types (CRDTs) emerged. CRDTs guarantee eventual consistency through mathematically proven commutative operations that can be applied in any order without requiring coordination .

While collaborative document editing addressed core synchronization issues, applying these techniques to source code editing introduces additional complexities. Source code differs from prose because of its strict syntactic structure, which must be preserved. Moreover, language-specific features such as autocompletion and refactoring demand that accurate abstract syntax trees be maintained in sync alongside the textCode .

Mirror has become a leading browser-based code editor framework, offering syntax highlighting, smart indenting, and a plugin architecture. Its transaction-based update model naturally supports collaborative editing. At the time of writing, Visual Studio Code Live Share exemplifies enterprise-grade collaborative coding integrated within a full-featured IDE. This tool allows developers to share not only their coding projects but entire workspaces, including terminals and debuggers. However, its reliance on Microsoft cloud infrastructure and installation requirements can make it feel cumbersome for quick collaboration sessions.

The cloud IDEs developed by browser-based platforms like Replit and CodeSandbox run only on browsers. However, the collaborative features in these platforms feel more like secondary features in IDEs with a full development environment. CodderPad and HackerRank are examples of the second type—which are optimized for technical interviews but not flexible for open-ended collaborative development. The technical infrastructure enabling real-time web applications underwent an unprecedented change with the establishment of the WebSocket protocol standard in 2011, allowing for the full-duplex communication. Socket.IO was developed on top of WebSockets, adding important features for production like: reconnecting automatically, acknowledging messages, routing based on namespaces, and using fallback when WebSockets are blocked. It has a room-based architecture that maps naturally to collaborative sessions. Users join named channels. A lot of issues arise when any user can run code safely. It is very hard to let users run any code safely. Modern code execution APIs like Judge0 and Piston utilise containerization to execute the code in isolated Docker containers which has strict resource limit. These services deal with the complicated infrastructure that manages specific language runtimes, which also ensure that malicious code does not affect other users or the host itself.

2.2 Existing Collaborative Systems

Collaborative software development is growing quickly and several systems have been developed to enable real-time communication between software developers. These solutions seek to support code sharing, communication and collaboration, but also vary greatly in

functionality, complexity and ease of use. It is important to understand these systems to determine their pros and cons and to develop a better system.

2.2.1 Cloud-Based Integrated Development Environments (IDEs)

Online Integrated Development Environments (IDEs) like web-based coding environments allow developers to code, compile and share code over the internet. These typically offer syntax highlighting, code debugging and project management. They offer collaboration tools, but their main emphasis is on providing a fully-fledged development environment rather than interaction. This means that their communication tools may be limited or need to be integrated with other services.

2.2.2 Real-Time Code Collaboration Tools

There are tools for collaborative programming that enables several users to work on the same codebase. These platforms have functionalities like multiple cursors, real-time updates, and collaboration sessions. But many of these tools are designed mainly for specific use cases such as coding interviews and pair programming, and do not support a wide range of development activities. Moreover, they don't always offer sophisticated file management and visualisation features.

2.2.3 Document-Based Collaboration Systems

Collaborative document editing in real-time synchronization applications is a good example of the power of real-time synchronization. Such systems use sophisticated algorithms for user synchronization. But they are not ideal for coding, since they don't support code-level editing, syntax highlighting, and management of multiple source code files.

2.2.4 Communication and Whiteboard Tools

Separate tools exist for communication (chat, video conferencing) and visualization (digital whiteboards). While these tools are highly effective individually, they require users to switch between multiple applications during collaborative sessions. This fragmentation reduces efficiency and disrupts workflow continuity.

2.2.5. Limitations of Existing Systems

Despite these advancements, still current collaborative systems suffer from several limitations:

- i. Lack of integration between coding, communication, and visualization tools
- ii. High setup complexity in some platforms
- iii. Limited real-time synchronization across multiple features

2.3 Real-Time Synchronization Techniques

Synchronization is one of the greatest challenges in all the collaborative systems, and it's very important for the data to be consistent for multiple users. In collaborative systems such as real-time code editors, where multiple users can edit the same editors, real-time collaboration and concurrency are a problem challenging. Several techniques have been developed to address this issue..

2.3.1. Operational Transformation (OT)

Operational Transformation (OT) is one of the first and most popular techniques for realtime collaboration. It involves transforming user operations (e.g., insertions and deletions) so they can be executed on all clients.

- i. Converges the document state across the users
- ii. Takes concurrent changes into account through operation transformation
- iii. Applied in systems such as collaborative word processors

But OT systems are complex due to the requirement for transformation functions and consistency. The need to handle a variety of edge cases in concurrent editing can make the system more complex.

2.3.2 Conflict-Free Replicated Data Types (CRDTs)

CRDTs are a relatively new technique that delivers eventual consistency without transformation of operations. In this model:

- i. Each copy of the data is held by a client
- ii. Changes are applied and then merged
- iii. Mathematical guarantees help ensure replicas converge

CRDTs are simpler to reason about in a distributed environment and scalable. They can, however, come with extra costs in terms of memory and complexity when applied to complex data structures such as source code.

2.3.3. Server-Based Event Sequencing

Alternatively, with systems such as Unified Code, event sequencing is coordinated by a server. In this model:

- i. All events are forwarded to a server
- ii. The server orders the events in sequence
- iii. The events are then sent back to all clients in order

This approach simplifies synchronization as there is only one source of events. It eliminates the need to deal with the complexity of OT and CRDTs, while still providing a consistent experience.

2.3.4. Web Sockets -Based Real-Time Communication

- i. Enables real-time data transfers without polling
- ii. Supports low-latency communication
- iii. Underpins real-time communications

2.3.5. Challenges in Synchronization

- i. Concurrent edits conflict resolution
- ii. Ensuring consistent operations in an unreliable network

- iii. Scaling to support more users
- iv. Latency and responsiveness

2.4 Communication Technologies (Web Sockets)

Collaborations in real-time communication are supported by communication technologies.

Traditionally, web communication has been based on the HTTP request-response model.

This works for web pages but not for real-time communication..

2.4.1. 1. Inefficiency of Traditional Methods

Initially, before the new protocols were invented, there were polling and long polling techniques to implement real-time communication:

Polling: The client sends frequent requests to the server for data.

2.4.2 WebSocket Protocol

The WebSocket protocol provides bi -directional communication.

Web Sockets have the following features:

- i. Open Connection: A connection is persistent, minimising overhead.
- ii. Bi-directional: Data is transferred in real-time.
- iii. Bidirectional: Can be pushed from the server or client.

2.4.3. Socket.IO Framework

Socket.IO provides

- i. Automatic Reconnection in case of network outages
- ii. Fallback (HTTP long polling) in case of no WebSockets support
- iii. Event-Driven Messaging for structured messages
- iv. Room Communication (users can join rooms)

In the Unified Code system, Socket.IO is used to provide real-time communication by putting users in rooms and broadcasting the actions of the collaboration (such as changes in the code, chat messages, whiteboard) to the users in the room.

2.4.4. Role in Real-Time Collaboration

Web Sockets and Socket.IO together do the RTC

- i. Real-time synchronization of code
- ii. Real-time chat communication
- iii. Real-time updates to whiteboards
- iv. Scalability to handle many users

2.4.5. Challenges

Although WebSocket systems are powerful, they have issues:

- i. Scalability with a large client base
- ii. Handling reconnecting clients
- iii. Data synchronization among clients

CHAPTER 3

SYSTEM DESIGN & METHODOLOGY

3.1 System Overview

Unified Code embraces a clear separation between the presentation client-side and the coordination on the server side, which are connected through a standardized event-driven protocol. This enables independent evolution of the client and server while ensuring a consistent contract. The architecture is broken down into three layers :
the client (browser based frontend), the coordination server (backend event hub) ,
and external services (code execution API).

3.1.1 Client-Side (Frontend)

- i. Developed using React.js for dynamic and responsive user interfaces
- ii. Includes modules like Code Editor (using CodeMirror 6, Whiteboard (using tldraw) ,Chat Panel, File Management System

3.1.2 Server-Side (Backend)

- i. Built using Node.js and Express.js
- ii. It Handles User connections and session management, Event processing and broadcasting and Synchronization of shared data

3.1.3 External Services

- i. Code execution handled via APIs (e.g., Piston API)

- ii. Ensures secure and isolated execution of user code

3.1.4 Communication Model

The system follows an **event-driven architecture** implies

- i. Processed and assigned a sequence order
- ii. Broadcast to other connected clients in the same room

3.1.5 Key Feature of System

- i. Real-time multi-user code editing
- ii. Integrated chat for communication
- iii. Collaborative whiteboard for visualization
- iv. Multi-file management system
- v. Code execution support for multiple languages
- vi. Low-latency synchronization across users

3.1.6 Design Principle

The system is built based on the following principles:

- i. Simplicity: Easy to use without complex setup
- ii. Modularity: Independent components for scalability
- iii. Performance: Low latency and efficient communication
- iv. Accessibility: Fully browser-based platform

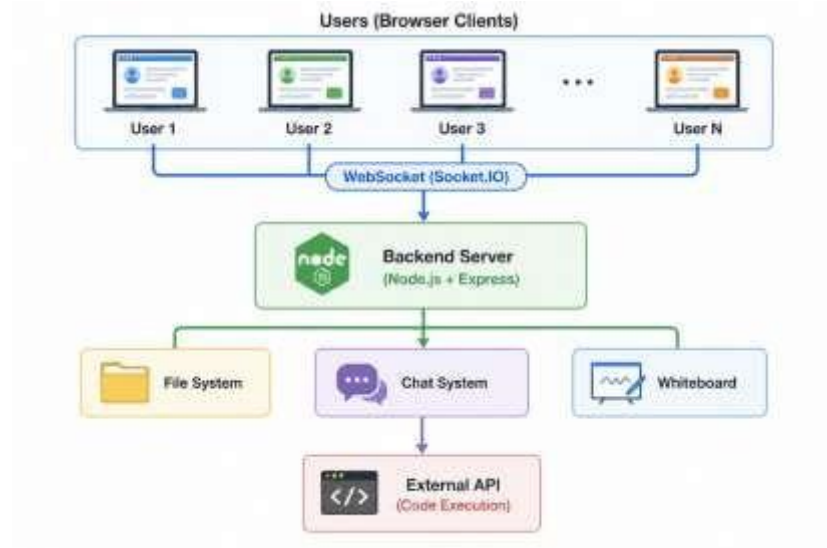


Figure 3.1: Overall System Architecture of Unified Code

3.2 Architecture Design

The fundamental architectural pattern is client-server with event-driven synchronization. Unlike traditional request response web applications, where clients explicitly request data and wait for responses, UnifiedCode employs bidirectional event streams where both clients and server asynchronously emit events representing state changes. Visual communication is an important aspect of collaborative programming when explaining algorithms or sketching architectures. Traditional whiteboard applications such as Miro are stand-alone platforms that require context switches. tldraw, launched in 2021, is part of a new generation of embeddable whiteboard components targeted for integration into web applications. An event-based API emits fine-grained drawing operations that can be transmitted across network connections and replayed on remote clients. Despite decades of improvements, state-of-the-art platforms have persistent limitations: fragmentation of features in which users need to keep several tools for different purposes; complexity barriers where installation and configuration are required; inaccessibility, where important features are tied to expensive licenses; heavyweight architectures that load a full fledged development environment for simple tasks; lack of customization possibilities; and neglect of education through oversimplification.

This event-centric design naturally models the collaborative interactions: when a user types a character, creates a file, or draws on the whiteboard, these actions become events that flow to the server and broadcast to other participants. Event-driven architecture offers several advantages for real time collaboration:

- i. Loose Coupling: Components interact through well defined events instead of method calls, enabling implementation and testing of components in isolation.
- ii. Natural Concurrency Model: All the events represent actions in temporal order, and it makes concurrent interactions easier.
- iii. Extensibility: In this, Additional features can be implemented by adding events and without changing the code.
- iv. Observable Behavior: The event logs provide complete records of collaboration sessions that are useful for debugging, auditing, and replaying.

3.3 Client Layer Architecture

The client side is a web application that can run in all major web browsers without plugins other than those required for HTML5/ES6. The client is written using React and uses a component based approach and each user interface component (editor, file tree, whiteboard) is a standalone React component with props and state

3.3.1 Component Architecture

The client employs a hierarchical component structure:

- i. App Component: The root component establishes global states that are user identity, WebSocket connection, and current room using React useContext hook.
- ii. Room Component: handles the main collaborative working area, which is divided into resizable panels. Handles room joining, user list management, and coordinates between sub components.

- iii. Editor Component: Wraps CodeMirror 6 and manages file content, syntax highlighting, and cursor synchronization. Emits edit events when local content changes; applies remote edit events as CodeMirror transactions.
- iv. File Tree Component: Displays the file tree with icons and is capable of performing CRUD operations on files/folders. It also tracks the currently selected file.
- v. Chat Component: It displays the message history, handles message input, and provides typing indicators. Messages are to be time-stamped, pulled for specific users.
- vi. Whiteboard Component: Embeds tldraw canvas, performs event-based synchronization of drawing operations. Provides a toolbar for selecting shapes and drawing mode.
- vii. Console Component: Provides the result of code execution including stdout, stderr, compilation error, and execution statistics: runtime, memory usage, exit code

3.3.2 CodeMirror 6 Integration

CodeMirror 6 is a new version of this widely used browser code editor that is modular, accessible and fast. Rather than a single editor, CodeMirror 6 is a collection of packages that can be plugged together to create a suitable editor.

Key features utilized in UnifiedCode include:

- i. Language Support: CodeMirror 6 has syntax highlighting for dozens of programming languages included via language packages (`@codemirror/lang-javascript`, `@codemirror/lang-python`, etc.). These bundles contain parsers that create the abstract syntax trees that are used by intelligent indentation and code folding.
- ii. Transaction System: All changes to the editor state, whether from user input or remote synchronization, are expressed as transactions themselves. This uniform model simplifies conflict resolution and enables undo/redo functionality.
- iii. Extension System: The facet and extension mechanism of CodeMirror 6 allows custom behaviors at various points in the editor lifecycle to be injected.

- iv. Accessibility: UnifiedCode provides accessibility because of its built-in ARIA annotations and keyboard navigation, which guarantee that the editor is compatible with screen readers and keyboard-only interaction

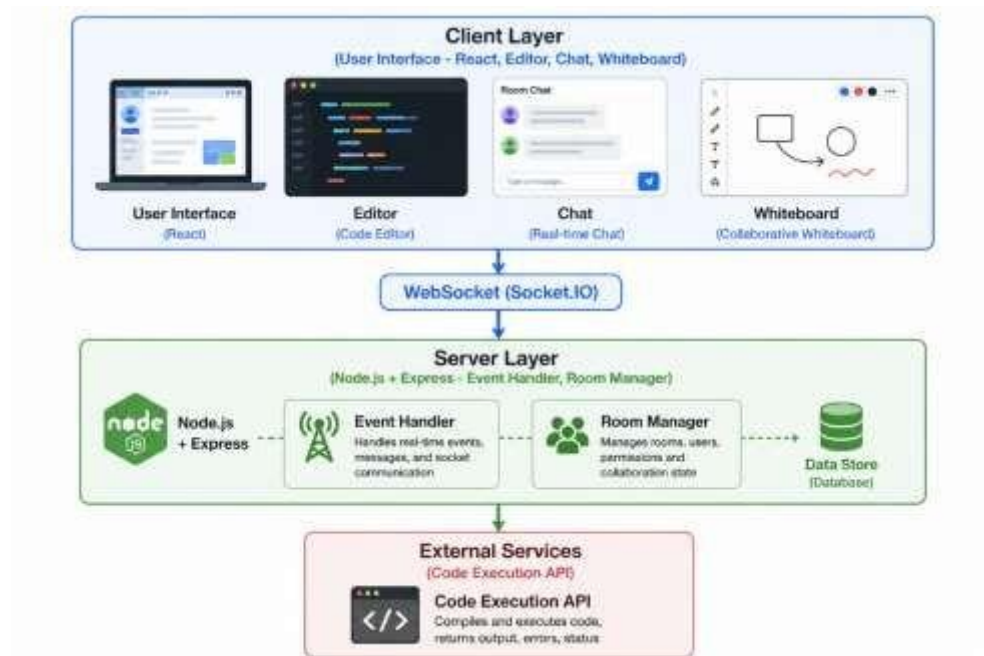


Figure 3.2: Layered Architecture of Unified Code System

3.4 Server Layer Architecture

The role of the server is to sequence and broadcast events: it provides a unique sequence for events, and distributes events to interested clients. This centralisation automates the ordering decisions that can arise in peer-to-peer collaboration.

3.4.1 Technology Overview

Node.js provides the runtime environment for JavaScript. Node is particularly well-suited for applications like UnifiedCode, where most time is spent waiting for network I/O- receiving events from clients and broadcasting to other clients-rather than performing CPU-intensive computation, due to its event-driven, non-blocking I/O model.

A single Node.js process is capable of handling thousands of concurrent WebSocket connections using minimal memory. Express.js handles the HTTP concerns: serving the static frontend bundle, providing health check endpoints, and can expose REST APIs for future features such as user authentication or session persistence

Socket.IO Server handles WebSocket connections and rooms. The server-side instance of Socket.IO mirrors the client-side API. `io.on('connection', callback)` is used for handling new connections while per-socket `socket.join(roomId)` assigns sockets to rooms. `io.to(roomId).emit(event, data)` broadcasts events to all sockets in a particular room.

3.4.2 State Management

The server maintains minimal ephemeral state:

- i. Room Registry: Maps room IDs to sets of connected sockets, used for targeted broadcasting.
- ii. User Registry: Maps sockets to user metadata, such as username, connection time, and last activity, for presence tracking.
- iii. Active File State: For each room, stores the current content of files in memory to support late-joining users who need to synchronize the initial state.
- iv. Chat History: Stores the most recent messages, such as the last 100, to allow new joiners to view the conversation context.

3.4.3 tldraw Integration

Unlike raster-based canvas drawing, tldraw uses SVG for crisp rendering at any zoom level and produces structured shape data, i.e, rectangles, ellipses, arrows, and texts. This structural representation enables efficient synchronization by transmitting only shape operations, i.e., create, move, resize, and delete, rather than full image data. tldraw's event system emits fine-grained updates, which UnifiedCode packages into events and broadcasts through Socket.IO. Remote drawing events are applied through tldraw's imperative API, which batch-updates shapes to maintain smooth rendering performance

3.4.4 Socket.IO Client Integration

Integration The socket.io-client library handles a WebSocket connection and events. On mounting the application, the client connects to the server, then joins a room (identified by URL token), and finally registers event handlers.

Room Isolation: Socket.IO rooms provide logical channels. Users joining the same room receive each other's events, while different rooms remain completely isolated. This architecture allows a single server to support many concurrent collaboration sessions.

3.5 Real-Time Collaboration Model

This synchronization model prioritizes consistency and simplicity for all features that ensure collaboration.

3.3.1 WebSocket Connection Management

Each client establishes a persistent WebSocket connection.

- i. Web Sockets are preferred with long polling as a fallback for restrictive networks.
- ii. Infinite automatic reconnection are essential for mobile users with low connectivity
- iii. React hooks manage the connection lifecycle and all the state synchronization after reconnection.

3.3.2 Room-Based Isolation

The Clients join the rooms after connection, creating logical channels that ensures:

- i. Isolation: The users in different rooms never receive each other's events.
- ii. Targeted Broadcasting: The event broadcast only to relevant room participants
- iii. State Synchronization: All the new joiners immediately receive the current room state using web sockets.

3.3.3 Event Emission Pattern

All collaborative actions through the emission follow a consistent pattern:

- i. Client applies changes locally immediately applies to the global state.
- ii. Client emits the events to the server with the change details and metadata of the change.
- iii. The server validates and broadcasts the change to all other room participants
- iv. All the remote clients receive and apply changes immediately, ignoring their own echoed events

3.3.4 Conflict Resolution

- i. The server handles all the requests in the order they arrive, and the second request fails politely with an error message in case of duplication.
- ii. The CodeMirror transaction system automatically merges changes immediately using offset-based positioning at high speed.
- iii. The clients detect the state divergence with the help of sequence numbers and request full state synchronization using the "last write wins" approach.

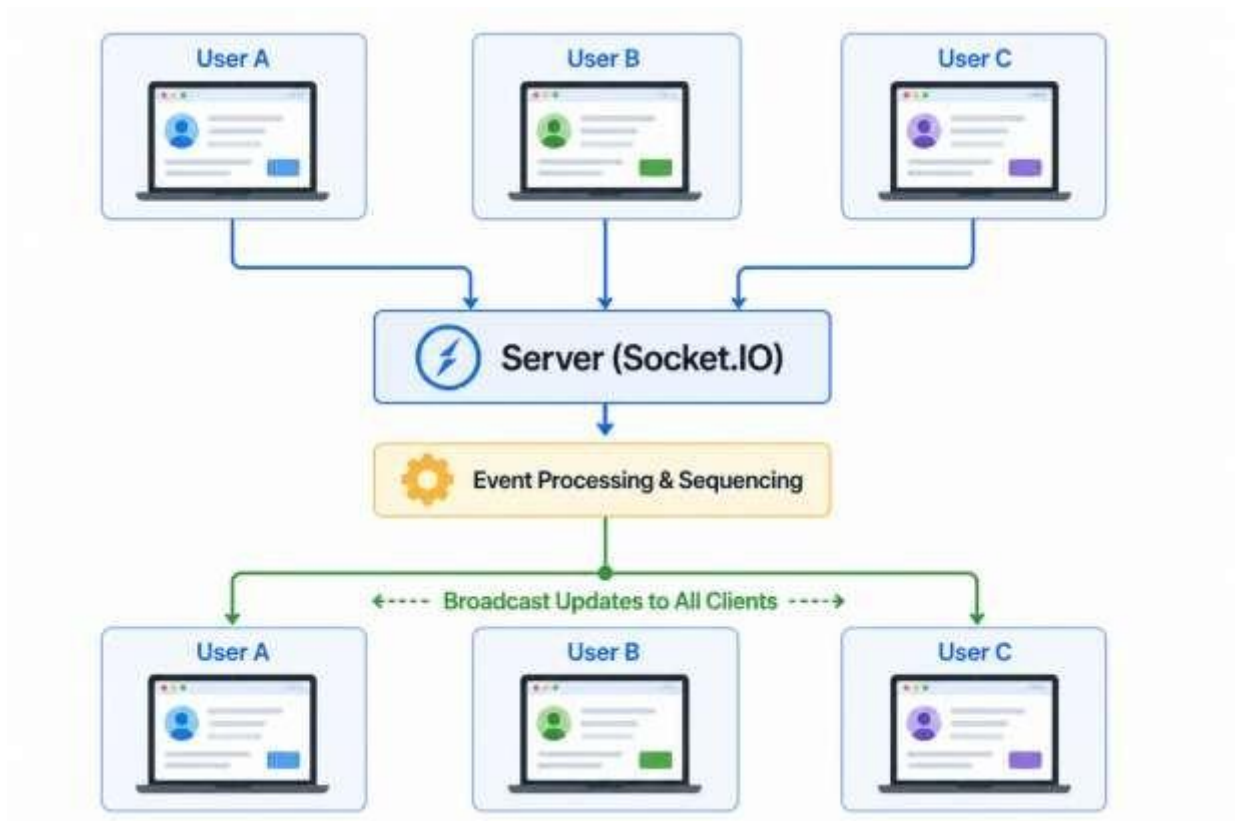


Figure 3.3: Real-Time Collaboration Event Flow

3.6 Collaborative Code Editing Implementation

3.6.1 Editor Initialisation

Each file gets a CodeMirror 6 instance with pre configuration:

- i. Language-specific syntax highlighting.
- ii. Update listeners for local edit detection.
- iii. Remote cursor extension displaying the locations of collaborators.

•

- iv. Support for custom themes and keymaps

3.6.2 Local Edit Detection

The CodeMirror's update listeners fire on every content change, capturing position, inserted text, deleted text, and cursor location, then it broadcasts it to the server.

3.6.3 Remote Edit Application

All the incoming edits from other users are applied as per CodeMirror transactions asap.

3.6.4 Remote Cursor Indicators

The Visual awareness is achieved through:

- i. Each user is assigned a unique color within the color theme itself.
- ii. Cursor widgets display the username in a colored flag at the cursor position.
- iii. Positions automatically adjust as the document changes.

3.6.5 Language Support

Many programming languages supported with auto-detection from file extensions and real time syntax highlighting

3.6.6 Advantages of Code Mirror Integration

- i. High performance and responsiveness
- ii. Modular and customizable design
- iii. Seamless real-time synchronization
- iv. Improved user experience with visual feedback

.

3.7 File Management System

- i. Tree Structure Files organized in a hierarchical tree with nodes containing: Unique ID, name, and type (file/folder) Content (files only) and children (folders only) Parent ID and auto-detected language
- ii. File Operations Create: Create new file (unique ID), update local file tree, broadcast to others Rename: Ensure no conflicts, update local, broadcast Delete: Ask if unsaved changes, close open editor if necessary, remove from tree, broadcast deletion..
- iii. File Selection: Selecting files changes the active editor by: Persisting current file's state Open new file editor Broadcast to others (presence indicators display coloured next to files they're viewing).
- iv. Upload/Download Upload: FileReader API imports local files into the editor Download: Creates Blob objects for individual file download or JSZip for entire project download as ZIP

3.8 Multi-Language Code Execution

3.8.1 Execution Request Flow

- i. Frontend initiates execution with language, code, and optional stdin.
- ii. Backend proxies request to the Piston API with appropriate language configuration
- iii. Piston executes code in an isolated environment Results (output, errors, runtime, memory, exit code) returned to the frontend.
- iv. Output displayed in the console panel

3.8.2 Language Detection

Language Detection File extensions automatically map to the appropriate language/compiler from 40+ supported languages

3.8.3 Output Display Console panel shows

Standard output and error messages Syntax highlighting for error patterns Execution statistics (runtime in ms, memory in KB) Exit code and signals

•

3.8.4 Stdin Support

A Dedicated text area allows users to provide standard input for programs requiring interactive input.

3.9 Integrated Text Chat

3.9.1 Message Structure

Messages contain ID, user ID, username, content, timestamp, and type (regular message or system notification)

3.9.2 Message Flow

- i. User types and sends a message
- ii. Message added to local chat immediately
- iii. Broadcast to all other room participants
- iv. Remote users receive the message, and add to history
- v. Notifications are shown if the chat panel is hidden, and the sound plays

3.9.3 Typing Indicators

- i. Emitted when the user begins typing
- ii. Cleared after 1 second of inactivity
- iii. Displayed as animated dots with typing user names

3.10 Integrated Text Chat

3.10.1 System of Themes Supported color themes include

Solarized, monokai, dracula, light, and dark updates all of the interface's CSS variables. changes the CodeMirror theme settings that are stored in localStorage

3.10.2 Customization of Layout Adaptable workspace configuration

- i. Panels that can be resized (file tree, editor, console/chat, whiteboard)
- ii. Usable layouts are prevented by minimum size restrictions.
- iii. Switch the visibility of the panel.
- iv. Auxiliary panel tabbed interface Across sessions, the layout is saved and restored

CHAPTER 4

IMPLEMENTATION & RESULT

4.1 Introduction

A comprehensive evaluation of UnifiedCode assessed multiple dimensions, i.e, real-time synchronization performance, resource utilization, consistency guarantees, and user experience. The Testing was conducted on typical desktop hardware to establish baseline performance characteristics as per pseudo real-time conditions.

4.2 Real-Time Synchronization Latency

The latency measures the time from when a user performs an action till the other users observe that action reflected in their interface

Event Type	2 Users (ms)	5 Users (ms)	10 Users (ms)
Code Edit	50–80	120–150	180–250
Cursor Movement	40–70	100–140	160–220
Chat Message	15–30	40–60	70–100
Drawing Update	60–90	150–220	250–350
File Operation	45–75	110–160	170–240

Table 1. Latency Comparison Table

The latency results demonstrates that UnifiedCode provides real-time performance for all scenarios. For the main use case (2-5 users connect), the latency is less than 150ms for most operations, which is imperceptible to the user with a reaction time of 200-300ms [1].

For under stress conditions with 10 users, latencies stay below 350ms i.e, below threshold.

4.2 Consistency Verification

The Consistency testing verified that all the clients get to identical states despite concurrent modifications.

Test Run	User Count	Total Edits	Hash Matches
1	3	487	3/3
2	5	823	5/5
3	3 (packet loss)	512	3/3
4	10	1,247	10/10

Table 2. Consistency Measuring Table

4.3 Resource Utilization

The Resource monitoring using the task managers assessed the computational overhead of real-time collaboration on both client and server.

User Count	CPU (%)	Memory (MB)	Network (KB/s)
2	10–15	150	10–20
5	25–40	180	40–70
10	45–60	220	120–180
20 (stress)	75–85	280	350–450

Table 3. Client Side Resource Utilization Table

Metric	Idle	Light Activity	Heavy Activity
CPU Usage (%)	3–5	8–15	20–35
Memory (MB)	120–150	180–220	280–350
Network RX (KB/s)	1–2	5–12	25–45
Network TX (KB/s)	0.5–1	3–8	15–30
Metric	Idle	Light Activity	Heavy Activity

Table 4. Server Side Resource Utilization Table

4.4 Feature Comparison

We compared Unified Code with three popular collaborative coding platforms: Replit, CodePen, and CoderPad. Unified Code offers a unique feature set, especially with the combination of whiteboard, extensive customizability, and being free. While platforms like Replit provide more feature complete IDE features-including file systems and version control- Unified Code focuses specifically on the collaborative coding session experience, making it simpler and more accessible for its target use cases..

Feature	UnifiedCode	Replit	CodePen
Real-Time Collaboration	✓	✓	Limited
Integrated Chat	✓	✓	✗
Whiteboard	✓	✗	✗
Multi-Language Execution	✓ (40+)	✓ (50+)	Limited (3)
Browser-Only (No Install)	✓	✓	✓
Multi-File Support	✓	✓	Limited
Theme Customization	✓	Limited	✓
Free Tier	✓	Limited	Limited

Table 5. Feature Comparison Table

CHAPTER 5

CONCLUSION & SCOPE

5.1 Conclusion

The Unified Code - Real-Time Collaborative Code Editor and Whiteboard project has addressed the challenges in modern collaborative coding.

With the shift in work to be more distributed, with people being in different locations and technical meetings being online, there is a need for an integrated system. The project offers a multi-functional integrated system.

The project is implemented using the latest web technologies including React.js, Node.js, Express.js and Socket.IO for real-time communication, enabling communication and coordination.

The system is interactive and user-friendly, utilizing the CodeMirror 6 code editor alongside the tldraw whiteboard for an enhanced experience. Additionally, by leveraging external APIs for code execution, it supports multiple programming languages without requiring local installations. A major achievement of the project is its real-time collaboration capability. The event-driven model broadcasts all user actions—such as code edits, chat messages, file operations, and whiteboard interactions—to all participants instantly. This ensures a consistent and seamless user experience, which is essential for effective collaboration. The system also demonstrates strong performance, having been tested to exhibit low latency, high consistency, and efficiency even in multi-user environments. Its modular, scalable, and extensible design allows it to be adapted for various applications. Moreover, the project successfully unifies different tools by integrating programming, communication, and visualization into a single platform. This integration streamlines the development process, enhances productivity, and eliminates the need for task switching. Overall, the Unified Code platform has proven to be a powerful, fast, and user-friendly collaborative environment. It demonstrates the use of

realtime communication technologies and provides an opportunity for future research and development in collaboration systems.

5.2 Future Scope

While the Unified Code system has been successfully built to offer a real-time collaborative environment with some integrated features, there are some potential improvements and extensions. Potential enhancements can include scalability, collaboration and user interface to make the system more robust and applicable to large systems.

5.2.1 Scalability Enhancements

The system can be used by small to medium sized user groups. For future scalability improvements:

- i. Using distributed server model
- ii. Using load balancing for users
- iii. Using cloud deployment for availability
- iv.

This will allow the system to be used by a large number of users and corporation.

5.2.2 Storage and Version Control

Future work can include:

- i. Support for databases (e.g., MongoDB) to store project and user data
- ii. Version control (e.g. Git)
- iii. File versioning

This will make more stable and allow for long-term use.

5.2.3 Advanced Collaboration

The system can be enhanced with other features like:

- i. Audio and video communication
- ii. User role (admin, editor, viewer) management
- iii. Conflict resolution (OT or CRDT)

This will aid in collaboration and synchronisation.

5.2.4 Improvisation in Code Editor

The code editor can be improved in the future in addition of :

- i. Language Server Protocol (LSP)
- ii. Code suggestions and error detection
- iii. Code formatter

5.2.5 Security Improvisations

Security can be improved by:

- i. Using authentication and authorization
- ii. Using encryption protocols
- iii. Preventing injection and other attacks This will make the product secure

5.2.5 External Tools

We will use other tools:

- i. GitHub or GitLab (version control)
- ii. CI/CD (continuous integration and continuous delivery)
- iii. Data storage with cloud storage

REFERENCES

- [1] N. Fraser, "Differential Synchronization," Jan. 2009. [Online]. Available: <https://neil.fraser.name/writing/sync/>
- [2] C. Sun, X. Jia, Y. Zhang, Y. Yang, and D. Chen, "Achieving convergence, causality preservation, and intention preservation in real-time cooperative editing systems," *ACM Trans. Comput.-Hum. Interact.*, vol. 5, no. 1, pp. 63–108, Mar. 1998.
- [3] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in *Proc. 13th Int. Conf. Stabilization, Safety, and Security of Distributed Systems (SSS)*, 2011, pp. 386–400.
- [4] M. Kleppmann and A. Wiggins, "CRDTs: Consistency without concurrency control," *Commun. ACM*, vol. 62, no. 6, pp. 48–55, May 2019.
- [5] M. Haverbeke, "CodeMirror: In-browser code editor," 2020. [Online]. Available: <https://codemirror.net/>
- [6] Microsoft, "Visual Studio Live Share," 2025. [Online]. Available: <https://visualstudio.microsoft.com/services/live share/>
- [7] Replit Team, "Replit: Build software faster," 2025. [Online]. Available: <https://replit.com/> Dec.
- [8] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, 2011. <https://tools.ietf.org/html/rfc6455> [Online]. Available:
- [9] G. Rauch, "Socket.IO: Realtime application framework," 2010. [Online]. Available: <https://socket.io/>

- [10] H. Gaspar, "Judge0: Open-source online code execution system," 2018. [Online]. Available: <https://judge0.com/>
- [11] Engineer Man, "Piston: High performance general purpose code execution engine," 2020. [Online]. Available: <https://github.com/engineer-man/piston>
- [12] S. Petros, "tldraw: A tiny little drawing app," 2021. [Online]. Available: <https://tldraw.com/>
- [13] R. B. Miller, "Response time in man-computer conversational transactions," in Proc. AFIPS Fall Joint Computer Conf., 1968, pp. 267–277.
- [14] J. Nielsen, "Usability Engineering," Academic Press, 1993.
- [15] K. Jahns, "Yjs: A CRDT framework for building collaborative applications," 2020. [Online]. Available: <https://yjs.dev/>
- [16] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," Commun. ACM, vol. 21, no. 7, pp. 558–565, Jul. 1978.
- [17] C. Sun and C. Ellis, "Operational transformation in real time group editors: Issues, algorithms, and achievements," in Proc. ACM Conf. Computer Supported Cooperative Work (CSCW), 1998, pp. 59–68.
- [18] P. Bailis and A. Ghodsi, "Eventual consistency today: Limitations, extensions, and beyond," Commun. ACM, vol. 56, no. 5, pp. 55–63, May 2013.